

COMPILER DESIGN · SYNTAX ANALYSIS

Parse Bench

Formal Grammar Reference

Prepared by **Mahimaa Priyadharshini**

Grammar-aware syntax analyzer · LL(1) recursive-descent parser

Repository: **Parse_Bench**

Date: **1 September 2026**

1 Overview

Parse Bench implements the syntax-analysis stage of a compiler for a small, restricted, C-like language. Given a stream of tokens (no lexer is included by design), it builds an LL(1) parsing table and a recursive-descent parser directly from a formal grammar, constructs a parse tree, and detects and recovers from syntax errors instead of stopping at the first one.

This document defines that grammar precisely: the surface form a person reads, the pure-BNF form the algorithms actually run on, the complete list of terminal and non-terminal symbols, and the FIRST/FOLLOW sets computed for every rule. All FIRST/FOLLOW values below were generated directly from the project's own `grammar.py`, not derived by hand, so they are guaranteed to match the running code.

2 Surface Grammar (EBNF)

This is the language as a human reads it, using `*` (zero or more), `?` (optional), and `|` (alternation):

```
program      -> statement*
statement    -> declStmt | assignStmt | ifStmt | whileStmt | block | printStmt
declStmt     -> "int" ID ";"
assignStmt   -> ID "=" expr ";"
ifStmt       -> "if" "(" cond ")" block ( "else" block )?
whileStmt    -> "while" "(" cond ")" block
block        -> "{" statement* "}"
printStmt    -> "print" "(" expr ")" ";"
cond         -> expr relop expr
relop        -> "<" | ">" | "<=" | ">=" | "==" | "!="
expr         -> term (("+" | "-" ) term)*
term         -> factor (("*" | "/" ) factor)*
factor       -> ID | NUM | "(" expr ")"
```

Rule by rule

- **program** — zero or more statements, one after another.
- **statement** — exactly one of: a declaration, an assignment, an `if`, a `while`, a `{ ... }` block, or a `print`.
- **declStmt** — `int`, then a name, then `;` (e.g. `int x;`)
- **assignStmt** — a name, `=`, an expression, `;` (e.g. `x = 1 + 2;`)
- **ifStmt** — `if (condition) block`, with an optional `else block`.
- **whileStmt** — `while (condition) block`.
- **block** — `{`, any number of statements, `}`.
- **printStmt** — `print(expression);`
- **cond** — `expression relop expression` (e.g. `x < 10`)
- **relop** — one comparison operator: `<` `>` `<=` `>=` `==` `!=`
- **expr** — one or more terms joined by `+` `/` `-`, left to right.
- **term** — one or more factors joined by `*` `/` `/`, left to right.
- **factor** — a name, a number, or a parenthesized expression.

3 Pure-BNF Grammar

The classic FIRST/FOLLOW/LL(1)-table algorithms are defined over pure BNF, not EBNF, so the `*` and `?` operators above are expanded by hand into plain, right-recursive rules with explicit epsilon (ϵ — "produces nothing") productions. This is the exact `GRAMMAR` dictionary defined in `grammar.py`:

```
program      -> statement program |  $\epsilon$ 

statement    -> declStmt
              | assignStmt
              | ifStmt
              | whileStmt
              | block
              | printStmt

declStmt     -> "int" ID ";"

assignStmt   -> ID "=" expr ";"

ifStmt       -> "if" "(" cond ")" block elsePart

elsePart     -> "else" block |  $\epsilon$ 

whileStmt    -> "while" "(" cond ")" block

block        -> "{" stmtList "}"

stmtList     -> statement stmtList |  $\epsilon$ 

printStmt    -> "print" "(" expr ")" ";"

cond         -> expr relop expr

relop        -> "<" | ">" | "<=" | ">=" | "==" | "!="

expr         -> term exprTail

exprTail     -> "+" term exprTail | "-" term exprTail |  $\epsilon$ 

term         -> factor termTail

termTail     -> "*" factor termTail | "/" factor termTail |  $\epsilon$ 

factor       -> ID | NUM | "(" expr ")"
```

What changed, and why

EBNF shorthand	Pure-BNF rewrite	Reason
<code>statement*</code> in <code>program</code>	<code>program</code> $\rightarrow$ <code>statement program</code> ϵ	right-recursion + explicit "or nothing"
<code>statement*</code> in <code>block</code>	new rule <code>stmtList</code> $\rightarrow$ <code>statement stmtList</code> ϵ	same idea, factored out since <code>block</code> also needs the braces
<code>("else" block)?</code>	new rule <code>elsePart</code> $\rightarrow$ <code>"else" block</code> ϵ	"optional" becomes "or nothing"
<code>(("+" "-") term)*</code>	new rule <code>exprTail</code> $\rightarrow$ <code>"+" term exprTail</code> <code>"-" term exprTail</code> ϵ	repetition becomes right recursion
<code>(("*" "/") factor)*</code>	new rule <code>termTail</code> $\rightarrow$ <code>"*" factor termTail</code> <code>"/" factor termTail</code> ϵ	same

No left recursion exists anywhere in the grammar — left recursion would break a top-down/recursive-descent parser — and every rule was already written this way by design; nothing had to be *eliminated*, only the repetition operators expanded.

4 Symbols

4.1 Terminals (tokens)

Category	Terminals
Keywords	INT (int), IF (if), ELSE (else), WHILE (while), PRINT (print)
Identifiers / literals	ID, NUM
Operators	ASSIGN (=), PLUS (+), MINUS (-), STAR (*), SLASH (/)
Relational operators	LT (<), GT (>), LE (<=), GE (>=), EQ (==), NE (!=)
Punctuation	LPAREN ((), RPAREN ()), LBRACE ({), RBRACE (}), SEMI (;)
End marker	EOF

4.2 Non-terminals

program (*start symbol*), statement, declStmt, assignStmt, ifStmt, elsePart, whileStmt, block, stmtList, printStmt, cond, relop, expr, exprTail, term, termTail, factor

17 non-terminals in total.

5 FIRST Sets

FIRST(X) = every terminal that could be the very first token of something X produces (ϵ means X can also produce nothing). Computed by the fixed-point algorithm `compute_first_sets()` — not hand-filled.

Non-terminal	FIRST set
program	ϵ , ID, IF, INT, LBRACE, PRINT, WHILE
statement	ID, IF, INT, LBRACE, PRINT, WHILE
declStmt	INT
assignStmt	ID
ifStmt	IF
elsePart	ϵ , ELSE
whileStmt	WHILE
block	LBRACE
stmtList	ϵ , ID, IF, INT, LBRACE, PRINT, WHILE
printStmt	PRINT
cond	ID, LPAREN, NUM
relop	EQ, GE, GT, LE, LT, NE
expr	ID, LPAREN, NUM
exprTail	ϵ , MINUS, PLUS
term	ID, LPAREN, NUM
termTail	ϵ , SLASH, STAR
factor	ID, LPAREN, NUM

6 FOLLOW Sets

FOLLOW(X) = every terminal that could legally appear immediately after an X (only meaningful for non-terminals; EOF marks "end of input"). Computed by the fixed-point algorithm `compute_follow_sets()`.

Non-terminal	FOLLOW set
program	EOF
statement	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
declStmt	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
assignStmt	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
ifStmt	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
elsePart	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
whileStmt	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
block	ELSE, EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
stmtList	RBRACE
printStmt	EOF, ID, IF, INT, LBRACE, PRINT, RBRACE, WHILE
cond	RPAREN
relop	ID, LPAREN, NUM
expr	EQ, GE, GT, LE, LT, NE, RPAREN, SEMI
exprTail	EQ, GE, GT, LE, LT, NE, RPAREN, SEMI
term	EQ, GE, GT, LE, LT, MINUS, NE, PLUS, RPAREN, SEMI
termTail	EQ, GE, GT, LE, LT, MINUS, NE, PLUS, RPAREN, SEMI
factor	EQ, GE, GT, LE, LT, MINUS, NE, PLUS, RPAREN, SEMI, SLASH, STAR

7 LL(1) Parsing Table & Conclusion

`build_ll1_table()` combines FIRST and FOLLOW into a table of (non-terminal, next token) → production to use. For this grammar it fills **75 conflict-free entries**.

Proof of LL(1)-ness: if any table cell were ever written twice — a FIRST/FIRST or FIRST/FOLLOW conflict — table construction raises an error immediately rather than silently overwriting it. That the table builds without error is the proof that this grammar is genuinely LL(1): a parser can always decide which production to use by looking at just one token of lookahead, with no backtracking and no ambiguity.

The full 75-entry table is available at runtime as `grammar.LL1_TABLE`, and its live entry count is shown in the React app's "Grammar & LL(1)" tab. This table is exactly what `parser.py`'s recursive-descent functions were written to agree with by construction — each parser function corresponds to one non-terminal and dispatches on the current token using the same FIRST-set logic captured here.